Notes for C++

Section 1 : Programming basics

```
#include <iostream>
using namespace std;

int main() {
  cout << "Hello world!";
}
```

The program consists of two parts:

Part A – the framework. The framework sits around our instructions and looks like this:

```
#include <iostream>
using namespace std;

int main() {
  ...
}
```

The framework provides the structure for your computer program. Each component of the framework means something and we will discuss the components as we move through the course. Right now, we want to focus on our own instructions so, for now, we will use the framework in every program we create and we will discuss the details later.

Part B – the instructions. Inside the framework, we place our instructions. This program has only one instruction which is:

```
  cout << "Hello world!";
```

The instructions are placed inside the **curly brackets** (braces). Instructions in a program are called **statements**. A program can have many statements but this program has only one statement which is a cout statement. The statements are **indented** – leave some spaces (or press tab) at the left-hand margin. Much of our course consists of learning new statements.

The cout statement

The cout statement is used in a C++ program to display text on the screen. All text that is between the quotation marks is displayed on the screen. Try changing the text and see what happens.

```
#include <iostream>
using namespace std;

int main() {
  cout << "My name is ";
  cout << "Bilbo";
}
```

This program has two cout statements and it will display the following text on the screen:

```
My name is Bilbo
```

Each cout statement continues from where the last one finished. Note that there is a space between "is" and "Bilbo" because I carefully left a space after "is" in the first cout statement. If that

space was not there, then the final display would be:

```
My name isBilbo
```

The computer always displays ***exactly*** what is between the quotation marks (including spaces).

Displaying several lines (using endl or \n)

The above program showed how two cout statements ended up displaying only one line of output. But what if we want to display several lines of output? We use the special ***newline*** character. Let us write another program (which must be saved separately from the first two programs):

```
#include <iostream>
using namespace std;

int main() {
  cout << "Bilbo Baggins" << endl;
  cout << "Bag End" << endl;
  cout << "24 Bagshot Row" << endl;
}
```

Note that endl causes the output to move to a new line, so the output will look like this:

```
Bilbo Baggins
Bag End
24 Bagshot Row
```

In place of endl, we can also use \n to move to a new line. This is often more convenient as it is included within the text. In this case, the program could look like this:

```
#include <iostream>
using namespace std;

int main() {
  cout << "Bilbo Baggins\nBag End\n24 Bagshot Row";
}
```

Practice using cout, endl and \n by writing your own programs.

File names and Saving programs

Each program must be saved separately **in its own file**.

Do not attempt to save more than one program together in the same file.

Each file must have a name. Standard file names contain letters and numbers but not any other strange characters. Standard file names do not contain spaces. On some operating systems you can include spaces in the name but the real name of the file will not be the name with the spaces. You can have problems when a file name is not the actual file name so it is a good habit to use standard file names.

Note that every program file name ends with the `.cpp` extension to show that it is a C++ program. If you are using Windows it is important to make the file extensions visible. For instructions on how to do this, google “windows 10 unhide file extensions”.

Save your files on any handy storage device – we usually use a USB flash drive but you could use a hard drive or any other device that is available for saving files. Massey may make storage space available for you but it is always good to have your own backup copy on your own USB.

You must PRACTICE your programming!!

Type in a program! Run the program.

Change the program and run it again. Note how the ***output*** changes.

Extra online practice (only if you want to, this is not part of our course)

This is a good website to help you to learn C++ programming:
https://www.learn-cpp.org/

We cover most of what is in "the Basics". Warning – this is not our course, it may cover things slightly differently from our course, it is only to get some extra practice.

Integer variables

- A variable is a location in the computer memory.
- Each variable has a name.
- Each variable has a type (for example an integer variable)
- Values are loaded into the variables.
- Each variable holds one value at a time but may change (vary) as the program runs.

Here is a program that defines and then uses integer variables:

```
#include <iostream>
using namespace std;

int value1, value2, result;

int main() {
  value1 = 3;
  value2 = 4;
  result = value1 + value2;
  cout << "The result is " << result;
}
```

This program ***defines*** three variables called `value1, value2` and `result`.
These are all integer variables (that is why `int` appears at the start of the definition).
The program loads the value 3 into variable `value1`.
The program loads the value 4 into variable `value2`.
The program adds `value1` to `value2` and loads the answer into variable `result`.
What value will be loaded into the variable `result`?
We display the result on the screen by using cout and the name of the variable.

An ***integer*** is a "whole number" that never has a fraction. It may be negative.
Examples of integers are: `64, -35, 0, 192`

Variable names

Each variable must have a unique name. You can select whatever name you like for your variables. However, it is very useful to give your variables meaningful names. This is not useful to the computer but it is useful to you (the programmer). It helps you to remember what each variable is being used for. In the example above, we used the name `result` for the variable that is loaded with the result of the calculation. Variable names must start with a letter of the alphabet (a to z). You can include digits (0 to 9) in the variable name but not as the first character in the name. You may not include other characters in the name and there must not be spaces in the name.

Assignment Statement

The assignment statement is used to load a number into a variable. For example: `value1 = 3;`
We talk about this as "`value1` is assigned the value of 3".
A variable can be loaded with a number, for example: `value2 = 4;`
A variable can be loaded with another variable, for example: `value2 = value1;`
A variable can be loaded with the result of a calculation, for example: `result = value1 + value2;`

Sequence

The ***order*** in which statements appear is important. Here is an example of the ***wrong order***:

```
result = value1 + value2;
value2 = 4;
value1 = 3;
```

This program will not calculate the correct result because `value1` and `value2` do not have the correct values when the calculation is performed. Try it out and see what happens.

Integer Arithmetic (arithmetic operators)

`a + b`	the add (or plus) operation
`a - b`	the subtract (or minus) operation
`a * b`	the multiply (or times) operation
`a / b`	the divide operation
`a % b`	the remainder (or modulus) operation

Integer Division

When integers are divided, the result is always an integer – there is no fraction.
For example:

The result of `6 / 3` is `2`
The result of `9 / 5` is `1`
The result of `2 / 3` is `0`

The Remainder Operator

The remainder operator returns what is left over after groups have been formed. For example:

The result of `9 % 4` is 1

In other words, we have 9 things and we remove groups of 4 (there are two groups) and then there is one thing left over (so the answer is 1). Other examples:

The result of `3 % 3` is `0`
The result of `9 % 5` is 4
The result of `2 % 3` is 2

BEDMAS

The usual priority rules are followed and brackets are calculated first. Some examples:

The result of `2 + 3 * 4` is `14`

But the result of `(2 + 3) * 4` is `20`

These operations are useful

These operations are useful in many calculations, and we will make use of them as we progress through the paper. You should practise using them with this small program:

```
#include <iostream>
using namespace std;

int x, y, result;

int main() {
  x = 23;
  y = 45;
  result = x % y;
  cout << "The result is " << result;
}
```

Change the numbers and the operation. Try and predict the result before running the program.

A Computer Program that does a useful calculation

Write a program to convert time in seconds to time in hours, minutes and seconds.

There are 60 seconds in a minute and there are 60 minutes in an hour.

For example: 65 seconds is 0 hours, 1 minutes, 5 seconds.

```
#include <iostream>
using namespace std;

int t, hours, minutes, seconds, allminutes;

int main() {
  t = 4527;
  allminutes = t / 60;
  hours = allminutes / 60;
  minutes = allminutes % 60;
```

```
  seconds = t % 60;
  cout << t << " seconds is " << hours << " hours ";
  cout << minutes << " minutes " << seconds << " seconds.";
}
```

Type this program in and run it. Understand what the program is doing.
Test the program with different times! Is it working correctly? How do you know?

A note on variables: I decided to use 5 variables – you can see them defined at the top of the program. You could use less than 5 variables (can you see how?) or you could use more than 5 variables. There are always different ways to write a program. Always give your variables meaningful names as it helps in understanding the program, especially it is a large program with many variables.

An important note on Testing

In order to test a program, you need to **know what the results will be before you run the program**.

For example, 4527 seconds is 1 hours, 15 minutes, 27 seconds

For example, 127 seconds is 0 hours, 2 minutes, 7 seconds

You need to know this before you run the program. When you run the program, you can then check if it is producing the correct results. Set up several ***test cases***.

Do *not* use the computer to provide you with unknown results!! There is no way to check if the calculations are being carried out correctly.

Always test with simple data!!

Input (also known as "reading" data)

We want to be able to enter values into a program.

```
#include <iostream>
using namespace std;

int value, result;

int main() {
  cin >> value;
  result = value + 3;
  cout << "The result is " << result;
}
```

Interesting things to notice in this program:

- the program will ***wait*** until a number is typed and enter has been pressed.
- while the program is waiting – ***nothing happens***. It is important to use a ***prompt*** (see below).
- the number that is typed is then loaded into the variable in the cin statement.

Use **prompts** that tell the user what to do as in this example:

```
#include <iostream>
```

```
using namespace std;

int length, width, area;

int main() {
  cout << "This program calculates the area of a rectangle\n";
  cout << "Type the width of the rectangle and press enter ";
  cin >> width;
  cout << "Type the length of the rectangle and press enter ";
  cin >> length;
  area = length * width;
  cout << "The area of the rectangle is " << area;
}
```

A **prompt** is a cout statement, or several cout statements, that tell the user what data should be entered. This is important to prevent a blank screen while the computer waits for input.

We can read more than one number on a line, for example:

```
#include <iostream>
using namespace std;

int a, b, c;

int main() {
  cout << "Type in three numbers: ";
  cin >> a >> b >> c;
  cout << "The numbers are " << a << b << c;
}
```

There are several ways to enter the three numbers. Some possibilities are:

```
31 2 -51        (there are spaces between the numbers and Enter at the end)
31   2    -51   (there are tabs between the numbers and Enter at the end)
31
2
-51             (each number is followed by Enter)
```

(or any combination of the above)

Whitespace

The term whitespace is used for spaces of any kind in user input – the space, tab or Enter keys (in any combination). The system knows how many values it must read (due to the syntax of the cin statement) so it ignores the whitespace and waits until the correct number of values has been entered.

Exceptional characters inside the double quotes of a cout statement

The following characters are exceptions to the general rule that everything between the double quotes in a cout statement is displayed on the screen:

`\n` – the newline character (can be used instead of endl)

`\t` – the tab character

If we **want** to display " or \, we use an extra \ in front of " or \.

Thus in a cout statement:

\\ causes one backslash to appear on the screen

\" causes the quote symbol to appear on the screen

What would the following cout statement display on the screen?

```
cout << "I said, \"Hi Frodo,\nhow are you?\"";
```

Note:

Exceptional characters are also known as escape sequences. There are more escape sequences available and you can find out more about them here:
https://ciphertrick.com/c-beginners-introduction-to-escape-sequences/

Comments

A comment is a note to yourself or other programmers. The computer completely ignores the comments. We find it useful to leave notes in programs to explain what the program is doing or why we are performing a particular calculation.

There are two types of comments:

```
/* this comment has a beginning and an end */
// this type of comment continues to the end of the line
```

Here is an example of comments (a mix of both types) used to explain a program:

```
#include <iostream>
using namespace std;

/* This program converts a time in seconds to a time in hours,
   minutes and seconds  */

int t, hours, minutes, seconds, allminutes;

int main() {
  cout << "Enter a time in seconds ";
  cin >> t;
  allminutes = t / 60;         // convert the seconds to minutes
  hours = allminutes / 60;     // calculate the hours
  minutes = allminutes % 60; // minutes is what remains
  seconds = t % 60;           // final seconds after removing minutes
  cout << t << " seconds is " << hours << " hours ";
  cout << minutes << " minutes " << seconds << " seconds.";
}
```

When you submit an assignment, it is a good idea to identify yourself as the author. The best way to do this is to place your ID and your name in a comment at the top of your program. For example:

```
// 2301785 Bilbo Baggins
//
```

```
// This is my assignment to add two numbers together

#include <iostream>
using namespace std;

int value1, value2, result;

int main() {
  value1 = 3;
  value2 = 4;
  result = value1 + value2;
  cout << "The result is " << result;
}
```

Notes:

- the C comment (starts with `/*` and ends with `*/`) can comment out any number of lines
- the C++ comment (`//`) comments out until the end of a line. To comment out several lines with this symbol, you need to repeat it at the start of every line.
- you can use any type of comment in a C++ program

The Programming Process

When creating a new computer program, most experienced programmers will follow a process. This process may vary from one program to the next (and also from one programmer to the next). This is not meant to be a fixed rule of how to create software. Some general guidelines are presented below.

1. State the problem:

What needs to be done? Launch a space rocket, model climate change, etc. Write a general overview of what the program is about.

2. Understand the problem:

What is the data? What calculations must be done? Are there any hidden details? There usually are! It is important to start **generating examples**. Everything we do is based on examples. You need a separate example for each situation. For example, the situation with heavy rainfall, the situation with no rainfall, etc. For a complex problem (e.g. a climate change simulation), understanding the problem may take months and there may still be some missing pieces.

3. Construct a solution to the problem (the solution is often an algorithm):

Create the algorithm. Is it similar to an existing algorithm? No need to reinvent the wheel.
Apply the solution to **the examples** in section 2. Does it work? Get the answers!!
For big programs we may move backwards and forwards between sections 2 and 3.

4. Create the program:

Write your program. You do not need to write it in the same order as it will appear when it is complete. For example, it is difficult to think of every variable when you start writing. Add the variable definitions as you go. What input do you need? Design your display. What output do you need? Once again, **work from your examples**.

5. Test the solution using simple examples:

Run your program. Test it on **your examples**. Make sure you know what the answers will be before you run the program. Then run it and check that the correct answers are produced. Do not use computers to produce answers. You need to know what the answers are before you run the program. Make sure your program works for every possible situation that may arise.

The NZ Cash Example

1. The problem: The price on the item is $8.97 but I must pay cash of $9.00
The price on the item is $8.93 but I must pay cash of $8.90

2. Understand the problem: The amount is rounded to the nearest 10.

3. Construct a solution: We will work in cents, thus $8.97 will be written as 897.

Algorithm
Add 5 to the amount in cents (e.g. 897 + 5 = 902)
Divide the total by 10 using integer division (e.g. 902 / 10 = 90)
Multiply the result by 10 to get the final amount (e.g. 90 * 10 = 900)

[***Note – an algorithm is not a program***]

4. Create the program:

```
#include <iostream>
using namespace std;

int price, cash, temp;

int main() {
  cout << "NOTE: enter all amounts as cents.\n";
  cout << "For example, $8.97 is entered as 897\n";
  cout << "Enter the price on the item: ";
  cin >> price;
  temp = price + 5;
  temp = temp / 10;
  cash = temp * 10;
  cout << "The price is " << price;
  cout << " and you must pay " << cash;
}
```

5. Test the program with examples:

Example 1: the item costs 897. Add 5 to get 902. Integer divide by 10 to get 90. Multiply by 10 to produce the final result of 900. Check that it runs correctly.

Example 2: the item costs 893. Add 5 to get 898. Integer divide by 10 to get 89. Multiply by 10 to produce the final result of 890. Check that it runs correctly.

Note: it is important to have at least one example for each different situation.

For this program we have three different situations as follows:

Situation 1 – the situation in which the amount rounds up (e.g. $8.97 becomes $9.00)

Situation 2 – the situation in which the amount rounds down (e.g. $8.93 becomes $8.90)

Situation 3 – the situation where the amount ends in 5 (e.g. $8.95). At the retailers discretion?

What does our program do with situation 3? Is this what every retailer wants?

Make sure that you have covered every possible situation!!

There are some important features in this program:

– we often replace a value in a variable by using the value from the variable to calculate a new result and then loading the new result back into the same variable.
For example: `temp = temp / 10;`

– we have made use of a well-known **algorithm** for rounding, which is:
add 5, replace the result with itself divided by 10, multiply the new value by 10

Test the program with different amounts. Make sure that you have tested all three situations.

Situation 3 is "at the discretion of the retailer". Can we write a program that works for all retailers in NZ? Should the program ask "what must we do with 5 cents – round up or round down?"

Special challenge

** This is only for those who want to investigate more advanced programming:

A) We are not used to dealing with amounts in cents. Modify the program so that it displays:

```
The price is $8.97 and you must pay $9.00
```

In other words, display the dollar signs and the dots as would be expected.

B) Modify the program to handle situation 3 for all retailers.

Assignment statement shortcuts

If the same variable appears on both sides of an assignment, we can use a shortcut.

For example:

`a = a + 7;` we can use the shortcut: `a += 7;`
`b = b * 3;` we can use the shortcut: `b *= 3;`
`c = c % 6;` we can use the shortcut: `c %= 6;`
`temp = temp / 10;` we can use the shortcut: `temp /= 10;`
`cents = cents - 50;` we can use the shortcut: `cents -= 50;`

There are two special cases of even shorter shortcuts:

```
a = a + 1;          a += 1;          a++;
```

all produce the same result (i.e. they add 1 to the value of `a`) – this is known as ***increment***.

and

```
z = z - 1;          z +-= 1;         z--;
```

all produce the same result (i.e. they subtract 1 from the value of `z`) – this is known as ***decrement***.

You do not have to use shortcuts but you must know what they mean.

The very short shortcuts (e.g. `num++;` and `value--;`) are often used in loops – when we get on to writing loops later in the course.

Shortcuts are also known as augmented assignments or compound assignments. For more information: https://en.wikipedia.org/wiki/Augmented_assignment